

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

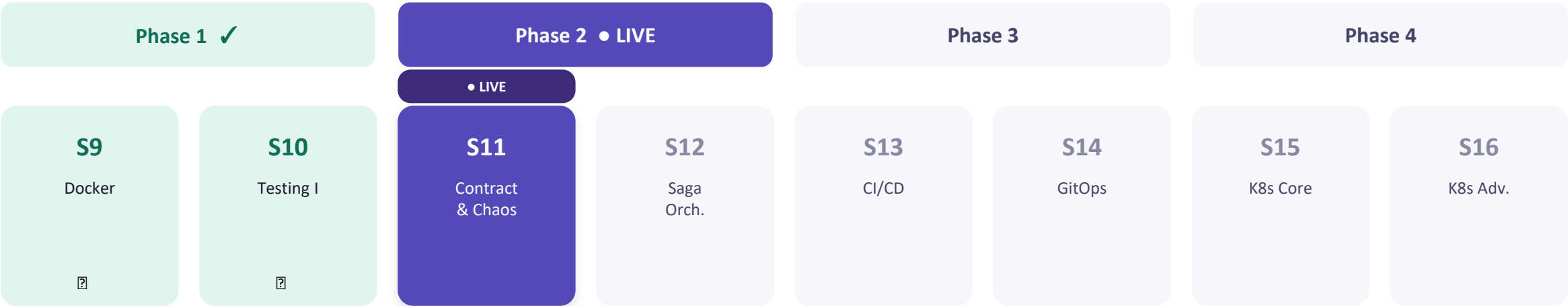


Session 11

Testing: Contract & Chaos

Pact Consumer-Driven Contracts · WireMock · Chaos Engineering

Phase 2 — Quality & Deployment



Phase 2 — Quality & Deployment · 8 sessions · S9 (Docker) and S10 (Testing I) complete — today we add Contract & Chaos testing

Full 29-session roadmap is in the Kickoff deck only.

We test the right layer now. But do services agree with each other?



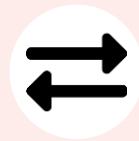
Session 10 — Solved

Unit tests isolate business logic.

@WebMvcTest isolates the controller.

TestContainers gives us a REAL PostgreSQL.

product-service now has a test at every layer.



Today — The Gap

All of these tests run **inside ONE service**.

None of them verify that order-service and inventory-service **AGREE** on the shape of the API between them.

That gap is where production breaks.

The Field Rename Incident

1

Order Service team mocks InventoryClient in their unit test. All tests pass. They ship.

2

Inventory Service team renames the response field: available → inStock. Their own unit tests still pass — they never call Order Service.

3

Both teams report 100% test coverage. Zero integration between the two test suites.

4

In production: Order Service receives inStock=true but deserializes available=null. Every order looks out of stock. All orders rejected.

Contract Testing: Consumer-Driven vs Provider-Driven



Provider-Driven

The provider publishes its API. Consumers must adapt to whatever shape the provider decides.

- ✗ Provider can silently break a consumer
- ✗ No enforcement mechanism
- ✗ Traditional API-first thinking



Consumer-Driven (Pact)

The consumer defines exactly what it expects. The provider **MUST** satisfy those expectations.

- ✓ Provider cannot break a consumer silently
- ✓ Violation caught by an automated test
- ✓ **Our choice for order-service ↔ inventory-service**

What Is a Contract Test

WITHOUT CONTRACT TESTS

Order Service tests:

- Mock InventoryClient → pass

Inventory Service tests:

- Unit tests → pass

Integration in production:

- Field renamed → null → orders **fail**

WITH PACT CONTRACT TESTS

Order Service tests:

- Pact consumer test → generates pact.json (contract file)

Inventory Service tests:

- Pact provider test → reads pact.json → verifies real API

If provider breaks the contract:

- Provider test **FAILS** at build time
← caught before shipping

🔍 Scope clarity:

- Pact tests are **NOT** end-to-end tests. They do **not** test business logic.
- They test ONE thing: does the **provider's API** shape match what the **consumer expects**?
- They are **fast** and run without a real running server.

The Pact Workflow

1. Order Service (Consumer) writes a Pact test:

```
'I expect GET /api/v1/inventory/check?productId=PROD-001&quantity=5'  
'to return HTTP 200 with body: {available: true, remainingStock: 95}'
```
2. Pact generates: `target/pacts/order-service-inventory-service.json`
3. Inventory Service (Provider) reads that file and verifies:

```
'Does my real API satisfy these expectations?'
```
4. If `inventory-service` renames `available` → `inStock`: provider test FAILS ☐

The Pact Workflow



If the provider renames a field the consumer depends on → Step 4 FAILS at build time, not in production.

Consumer vs Provider: Roles & Responsibilities

Concept	What It Means	In Our Platform
Consumer	The service that CALLS the API	order-service (calls Inventory)
Provider	The service that SERVES the API	inventory-service (serves /check)
Pact File	JSON file defining the expected interactions	target/pacts/order-service-inventory-service.json
Pact Broker	Shared server where pact files are published and teams share contracts	In this course: local file sharing (Broker is optional for teams)
Contract Violation	Provider test fails because real API no longer matches consumer expectations	Caught at test time, not production

Consumer vs Provider: Roles & Responsibilities

Concept	Consumer	Provider
Who is it in our platform?	order-service	inventory-service
What does it do?	Calls the API — via Feign	Serves the API — /check endpoint
What does it write?	@Pact interaction + @Test	@Provider verification test
Where does the pact.json live first?	target/pacts/ (generated here)	Read from consumer's output folder
What happens on violation?	N/A — consumer defines the truth	Verification test FAILS

Pact Consumer — Add Pact Consumer dependency

`pom.xml` — `order-service`

```
<dependency>
  <groupId>au.com.dius.pact.consumer</groupId>
  <artifactId>junit5</artifactId>
  <version>4.6.7</version>
  <scope>test</scope>
</dependency>

<!-- Pact files will be written to: target/pacts/ by default -->
```

Pact Consumer — Define the Interaction / Contract

`OrderServiceInventoryContractTest.java` — (in `order-service/src/test/`)

```
@ExtendWith(PactConsumerTestExt.class)
@PactTestFor(providerName = "inventory-service", port = "8888")
class OrderServiceInventoryContractTest {

    // Define the contract: what order-service expects from inventory-service
    @Pact(consumer = "order-service", provider = "inventory-service")
    RequestResponsePact checkStockAvailable(PactDslWithProvider builder) {
        return builder
            .given("PROD-001 has 100 units in stock")
            .uponReceiving("a stock check for PROD-001 quantity 5")
                .path("/api/v1/inventory/check").method("GET")
                .query("productId=PROD-001&quantity=5")
            .willRespondWith()
                .status(200)
                .body(LambdaDsl.newJsonBody(body -> body
                    .booleanValue("available", true)    // field name = the CONTRACT
                    .integerType("remainingStock", 95) // type matters, exact value doesn't
                ).build())
            .toPact();
    }
}
```

Pact Consumer — Verify Our Feign Client Against It

`OrderServiceInventoryContractTest.java` — (in `order-service/src/test/`) — Continue

```
// The test: does our Feign client correctly deserialize the contract response?
@Test
@PactTestFor(pactMethod = "checkStockAvailable")
void checkStock_deserializesAvailableField_correctly(MockServer mockServer) {

    // Pact starts a mock HTTP server at port 8888 for this test
    InventoryClient client = buildFeignClient(mockServer.getUrl());

    StockCheckResponse response = client.checkStock("PROD-001", 5);

    assertThat(response.available()).isTrue();
    assertThat(response.remainingStock()).isGreaterThan(0);
}

private InventoryClient buildFeignClient(String baseUrl) {
    return Feign.builder()
        .decoder(new JacksonDecoder())
        .target(InventoryClient.class, baseUrl);
}

} // Running this test generates: target/pacts/order-service-inventory-service.json
```

mvn test Generates the Contract File

```
$ mvn test -pl services/order-service -Dtest=OrderServiceInventoryContractTest
```

```
target/pacts/order-service-inventory-service.json
```

```
{
  "consumer": { "name": "order-service" },
  "provider": { "name": "inventory-service" },
  "interactions": [ {
    "description": "a stock check for PROD-001 quantity 5",
    "request": { "method": "GET", "path": "/api/v1/inventory/check", ... },
    "response": { "status": 200, "body": { "available": true, "remainingStock": 95 } }
  } ]
}
```

*This JSON file IS the **contract**. inventory-service will **verify** against it next.*

Pact Provider — Add Pact Provider dependency to inventory-service

`pom.xml` — `inventory-service`

```
<dependency>
  <groupId>au.com.dius.pact.provider</groupId>
  <artifactId>junit5spring</artifactId>
  <version>4.6.7</version>
  <scope>test</scope>
</dependency>
```

Pact Provider — Verify the Real API Satisfies It

`InventoryServicePactVerificationTest.java` — (in `inventory-service/src/test/`)

```
@Provider("inventory-service")
@PactFolder("../order-service/target/pacts")
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
class InventoryServicePactVerificationTest {

    @LocalServerPort
    private int port;

    @BeforeEach
    void setUp(PactVerificationContext context) {
        // Tell Pact where the real provider is running
        context.setTarget(new HttpTestTarget("localhost", port));
    }

    @TestTemplate
    @ExtendWith(PactVerificationInvocationContextProvider.class)
    void pactVerificationTestTemplate(PactVerificationContext context) {
        context.verifyInteraction();
    }
}
```

Pact Provider — Verify the Real API Satisfies It

`InventoryServicePactVerificationTest.java` — (in `inventory-service/src/test/`) — Continue

```
// Provider state setup — matches 'given(...)' in the Consumer test
@State("PROD-001 has 100 units in stock")
void setupProd001Stock() {
    // Ensure the inventory has PROD-001 with 100 units before the test runs
    // (Inventory uses in-memory map — reset it to the expected state)

    inventoryService.resetStock("PROD-001", 100, 0);
}

@Autowired
private InventoryService inventoryService;
}

// If inventory-service renames 'available' → 'inStock' in its response:
// This test FAILS with: 'Expected field available but was missing'
// ← The contract violation is caught HERE, not in production.
```

What happens when the Provider renames a field:

- Consumer expects: { "available": true, "remainingStock": 95 }
- Provider returns: { "inStock": true, "remainingStock": 95 }
- Result: Pact provider test FAILS — 'available field missing in response'
- Without Pact: test passes, production fails silently with available=null

What Pact Catches That Unit Tests Miss

The Field Rename, Replayed With Pact In Place

Inventory renames `available` → `inStock`. Its own unit tests still pass.

But the **Pact provider verification test** reads the consumer's pact.json, replays the exact request, and finds **"available" field missing**.

Unit tests (mocked)

❓ Pass — mock never sees real shape

Integration tests (S10)

❓ Pass — same service, no cross-boundary check

Pact provider test

❓ **FAILS — this is the safety net**

WireMock vs Mockito

Testing isolated logic, no I/O?

Mockito

Simulating an external HTTP service?

WireMock

Verifying the correct URL/method was called?

WireMock

Testing in-process Spring bean interactions?

Mockito

Your code uses OpenFeign/RestTemplate/WebClient?

WireMock — it makes real HTTP calls

Your code calls a @Service or @Repository?

Mockito — @Mock or @MockBean

CI pipeline, no real service available?

WireMock

Team new to testing microservices?

Start Mockito, add WireMock later

WireMock: A Real HTTP Server, Not a Java Mock



Mockito — mocks a bean

```
@Mock InventoryClient client;
```

Replaces the Java object entirely.

- ✗ Never sends real HTTP
- ✗ Never tests (de)serialization
- ✗ Never catches a URL typo

```
// Mockito mock — works for Java beans:
@Mock InventoryClient inventoryClient; // ← an interface,
Mockito generates a proxy
when(inventoryClient.checkStock("PROD-001", 5))
    .thenReturn(new StockCheckResponse("PROD-001", 5, true,
95));
// □ Tests business logic in OrderService
// □ Does NOT test that Feign sends the correct HTTP request
// □ Does NOT test JSON serialization/deserialization
// □ Does NOT test the correct URL path and query parameters
```



WireMock — a real server

```
@AutoConfigureWireMock(port = 0)
```

Starts an actual HTTP server on a random port.

- ✓ Feign/RestTemplate sends real HTTP
- ✓ JSON deserialization actually runs
- ✓ **verify()** checks the exact request sent

```
// WireMock stub — works for real HTTP calls:
stubFor(get(urlPathEqualTo("/api/v1/inventory/check"))
    .withQueryParam("productId", equalTo("PROD-001"))
    .withQueryParam("quantity", equalTo("5"))
    .willReturn(aResponse()
        .withStatus(200)
        .withHeader("Content-Type", "application/json")
        .withBody("{\"available\":true,\"remainingStock\":95}")));
// □ Tests that Feign sends GET
/api/v1/inventory/check?productId=PROD-001&quantity=5
// □ Tests that JSON deserialization produces the correct
StockCheckResponse
// □ Catches URL typos, wrong query params, missing headers
```

WireMock — Add WireMock dependency

`pom.xml` — `order-service`

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-contract-wiremock</artifactId>
  <scope>test</scope>
  <!-- spring-cloud-contract-wiremock wraps WireMock for Spring Boot integration -->
  <!-- No version needed: BOM manages it -->
</dependency>
```

WireMock integration test — Happy Path: Payment APPROVED

`OrderServicePaymentWireMockTest.java` — (in `order-service/src/test/`)

```
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@AutoConfigureWireMock(port = 0) // WireMock starts on random port
class OrderServicePaymentWireMockTest {

    @Autowired private OrderService orderService;

    // — Happy Path: Payment Service returns APPROVED —
    @Test
    void createOrder_returnsConfirmed_whenPaymentApproved() {
        // Stub payment-service to return APPROVED
        stubFor(post(urlEqualTo("/api/payments"))
            .willReturn(aResponse()
                .withStatus(200)
                .withBody("{\"transactionId\":\"TXN-001\",\"status\":\"APPROVED\"}")));

        OrderResponse response = orderService.createOrder(new OrderRequest("PROD-001", 1, new BigDecimal("100.00")));

        assertThat(response.status()).isEqualTo("CONFIRMED");
        assertThat(response.transactionId()).isEqualTo("TXN-001");
    }
}
```

WireMock integration test — Failure Path: 503 Triggers the Circuit Breaker

`OrderServicePaymentWireMockTest.java` — (in `order-service/src/test/`) — Continue

```
// — Failure Path: Payment Service returns 503 —
@Test
void createOrder_returnsPending_whenPaymentServiceUnavailable() {

    stubFor(post(urlEqualTo("/api/payments"))
        .willReturn(aResponse().withStatus(503)));    // Payment is down

    OrderResponse response = orderService.createOrder(
        new OrderRequest("PROD-001", 1, new BigDecimal("100.00")));

    // Circuit Breaker fallback (Session 4) should return PENDING
    assertThat(response.status()).isEqualTo("PENDING");
}

// CS-05 Failure Flow: this closes the loop opened in Session 4 —
// we CONFIGURED the fallback then, we VERIFY it here.
```

WireMock integration test — Correct HTTP call

`OrderServicePaymentWireMockTest.java` — (in `order-service/src/test/`) — Continue

```
// — Verify the correct HTTP call was made —
@Test
void createOrder_sendsCorrectPayload_toPaymentService() {

    stubFor(post(urlEqualTo("/api/payments"))
        .willReturn(aResponse().withStatus(200)
            .withBody("{\"transactionId\":\"TXN-002\",\"status\":\"APPROVED\",\"amount\":250.00}")));

    orderService.createOrder(
        new OrderRequest("PROD-002", 2, new BigDecimal("250.00")));

    // Verify WireMock received a POST with the correct amount
    verify(postRequestedFor(urlEqualTo("/api/payments"))
        .withRequestBody(matchingJsonPath("$.amount", equalTo("250.0"))));
}
```

Test Run: Circuit Breaker Opens, Fallback Returns PENDING

```
$ mvn test -Dtest=OrderServicePaymentWireMockTest

createOrder_returnsConfirmed_whenPaymentApproved          PASSED
createOrder_returnsPending_whenPaymentServiceUnavailable  PASSED
createOrder_sendsCorrectPayload_toPaymentService          PASSED

[WIREMOCK] 503 stub matched: POST /api/payments
[CB] paymentService circuit breaker recorded failure
[FALLBACK] Returning PENDING order – payment will retry

BUILD SUCCESS · 3 tests, 0 failures
```

WireMock proved order-service SENDS the right request. Pact (S8–S12) proves inventory-service SERVES the right response. Together: full contract coverage.

Chaos Engineering: Break It Before Production Does

“If you know your system will fail at 3 AM under load, why not break it deliberately at 2 PM when your team is awake?”



The Hypothesis

If inventory-service goes down, Circuit Breaker opens and orders return PENDING, not 500.



The Experiment

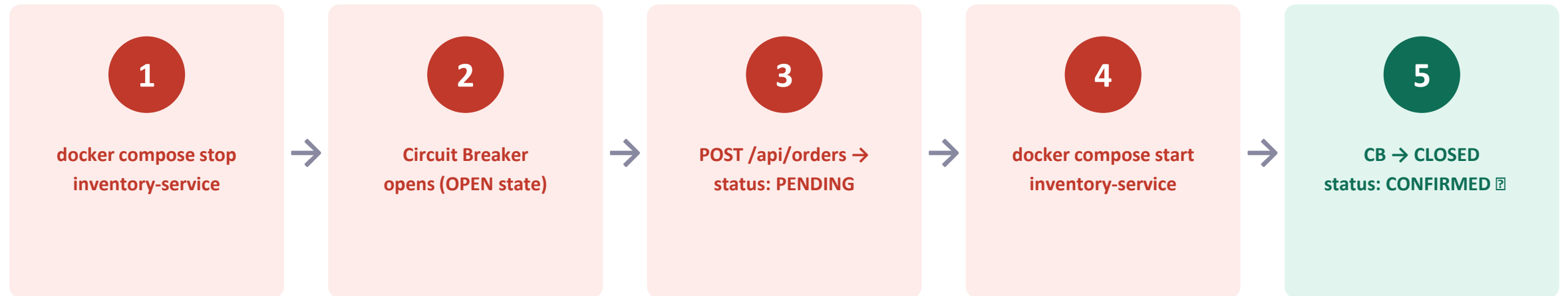
Stop the inventory-service container. Observe actual system behavior under real failure.



The Result

Did the platform degrade gracefully (PENDING) or catastrophically (500 / timeout / crash)?

Chaos Demo Flow — Stop, Observe, Recover



Beat 5 is intentionally teal, not coral — it is the recovery confirmation, the payoff of the whole resilience investment since Session 4.

Chaos Execution: Watch the Logs

```
$ docker compose stop inventory-service
inventory-service  Stopped

$ curl -X POST http://localhost:8080/api/orders -d '{...}'
{"status":"PENDING","message":"System busy – will retry"}

$ curl http://localhost:8082/actuator/circuitbreakers
"inventoryService": { "state": "OPEN" }

$ docker compose start inventory-service
$ curl -X POST http://localhost:8080/api/orders -d '{...}'
{"status":"CONFIRMED","transactionId":"TXN-..."}
```

Wrong outcome: HTTP 500 or a timeout — that means the resilience pattern from Session 4/5 is not actually working end-to-end.

Manual Chaos Demo — Closing the Loop from Phase 1

This demo verifies that the Circuit Breaker from Session 4 actually works in a real containerized environment — not just in a unit test.

```
# — SETUP: platform running in Docker (from Session 9) —————
docker compose up -d
# Verify all services healthy
docker compose ps

# — STEP 1: Confirm happy path —————
curl -X POST http://localhost:8080/api/orders \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer <test-jwt>' \
  -d '{"productId":"PROD-001","quantity":1,"amount":100.00}'
# Expected: {"status":"CONFIRMED", ...}

# — STEP 2: CHAOS — stop inventory-service container —————
docker compose stop inventory-service
# ← Hypothesis: Circuit Breaker opens, order returns PENDING (not 500)
```

Manual Chaos Demo — Closing the Loop from Phase 1

```
# — STEP 3: Place an order — what happens? —————
curl -X POST http://localhost:8080/api/orders \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer <test-jwt>' \
  -d '{"productId":"PROD-001","quantity":1,"amount":100.00}'
# Expected: {"status":"PENDING", "message":"System busy..."} ← CB fallback ☐
# Wrong:    HTTP 500 / timeout ← resilience pattern not working ☐

# — STEP 4: Verify Circuit Breaker state —————
curl http://localhost:8082/actuator/circuitbreakers
# Expected: inventoryService CB state = OPEN or HALF_OPEN

# — STEP 5: Restore and verify recovery —————
docker compose start inventory-service
# Wait ~10s for CB to transition OPEN → HALF_OPEN → CLOSED
# Place another order → Expected: {"status":"CONFIRMED"} again ☐
```

1. We designed the Circuit Breaker in Session 4 — today we PROVE it works under real failure
2. The WireMock test (above) tested it in code. This demo tests it in a running system.
3. Chaos Engineering is the final validation step: write tests, then BREAK the system to verify the tests are right
4. A passing Circuit Breaker test + a passing chaos demo = confidence in production resilience

Lab 9B — Contract Test + WireMock

20 min in-session · complete as homework · Builds on Lab 9A (Session 10)

1

Pact Consumer Test

order-service defines the contract → generates `target/pacts/order-service-inventory-service.json`

2

Pact Provider Verification

inventory-service verifies its real API satisfies the pact. Rename a field → watch it fail.

3

WireMock Test

Stub payment-service. Happy path → CONFIRMED. 503 → PENDING (Circuit Breaker fallback).

What We Added Today

 Capability Added

Contract Safety Net + Chaos Validation

Services cannot silently break each other's API contracts. Resilience patterns are proven under real failure, not just unit tests.

 Services Updated

- order-service** — Pact consumer + WireMock tests
- inventory-service** — Pact provider verification

Enterprise E-Commerce Platform — After Session 11: Contracts enforced, resilience validated under real failure



Docker



Testing I



Contract✓



Saga Orch.



CI/CD



GitOps



K8s Core



K8s Adv.



DAILY QUIZ

8 Questions · 10 Minutes

Pact · WireMock · Chaos Engineering

Homework & Checkpoint Commit

? Definition of Done — Session 11

- ✓ Pact consumer test generates pact.json
- ✓ Pact provider test passes against it
- ✓ WireMock happy + failure path tests pass
- ✓ mvn test — both services BUILD SUCCESS

Commit: `session-11: add-contract-tests-wiremock-order-inventory`

? Pre-Session 12 Knowledge Check

Q1: Difference between Choreography Saga and Orchestration Saga?

Q2: What does `@StartSaga` do in a Saga orchestrator and when does it trigger?

Q3: Why is an association value needed in a distributed Saga?



SESSION 11 COMPLETE

Contract Safety Net + Chaos Validation [?](#)

Next: Session 12 — Saga Orchestration (Spring Boot State Machine)

Monday, 3:00–5:30 PM · Online